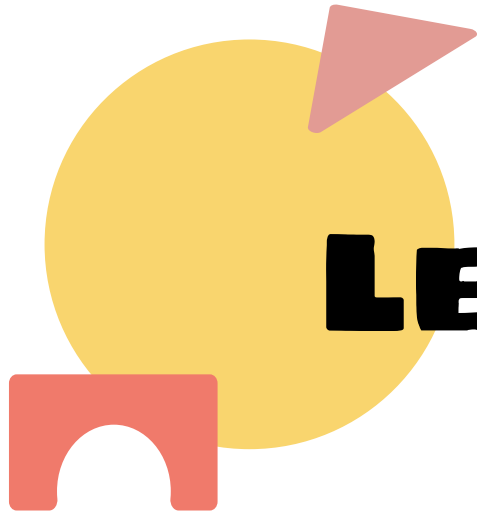




LEKTION 15

FACADE

MÅL

EFTER ENDT UNDERVISNING KAN DEN STUDERENDE:

- Redegøre for facade
- Udfærdige et eksempel og anvendelse af facade



AGENDA

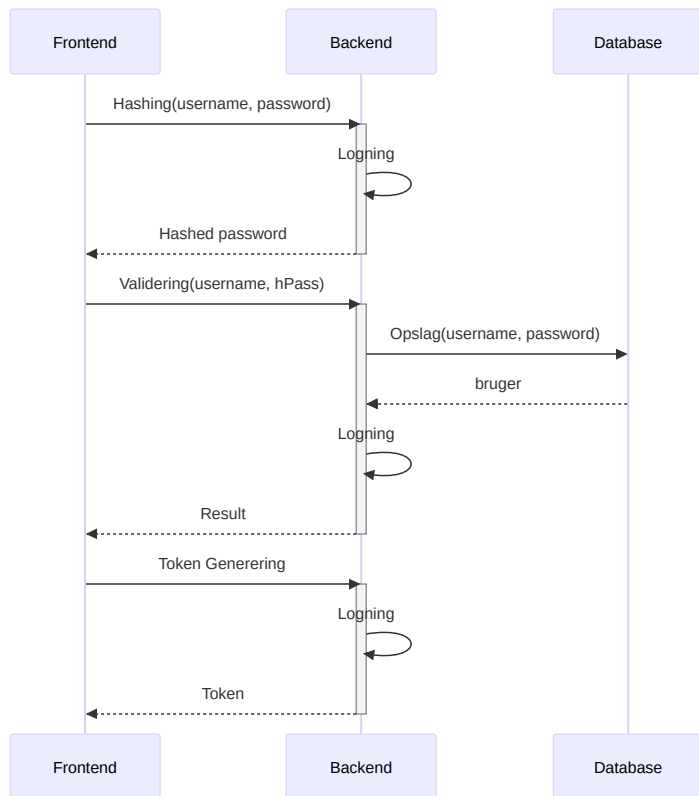
- 
1. Recap
 2. Beskrivelse
 3. Behov
 4. Løsning
 5. Opgave
 6. Næste gang

BESKRIVELSE

VI HAR ET SYSTEM HVOR DER ØNSKES BESKYTTET ADGANG, GRUNDET FØLSOMHED I DATA. FOR AT BESKYTTE ADGANG, SKAL DER LAVES ET LOGIN SYSTEM, SOM KRÆVER USERNAME OG PASSWORD. MEN FOR AT VÆRE SIKRE, SKAL USERNAME OG PASSWORD, BÅDE GENNEMGÅ SALT AND PEPPER, HASHED OG SÆTTES KRAV OM KODEORD POLITIK. DETTE KRÆVER AT PASSWORD OG USERNAME FØRST SKAL VALIDERES, DER SKAL LAVES HASHING, VED GODKENDELSE SKAL DER SENDES EN TOKEN RETUR.

BEHOVET

LØSNINGSFORSLAG



PROBLEMET

Problemer med denne tilgang

- Stærk kobling mellem frontend og validering.
- Hvis der er noget som skal ændres, kræver det mange tilpasninger.
- Security issue, hvis udstiller kontinuerlig data på vores kommunikations linje.

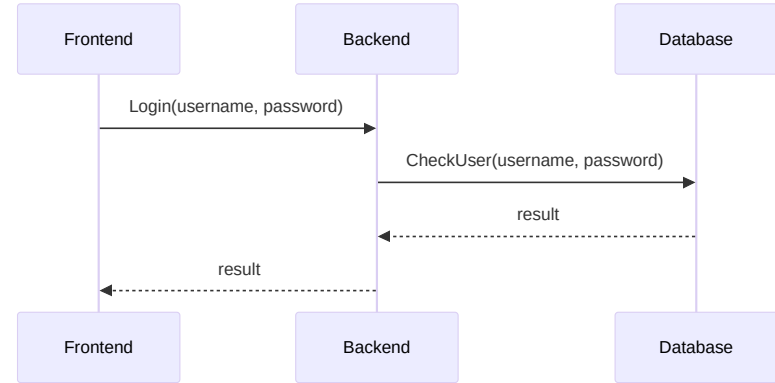
BEHOVET

BEHOV

Vi ønsker:

- Én simpel indgang
- Intern kompleksitet skjules
- Tydeligt interface

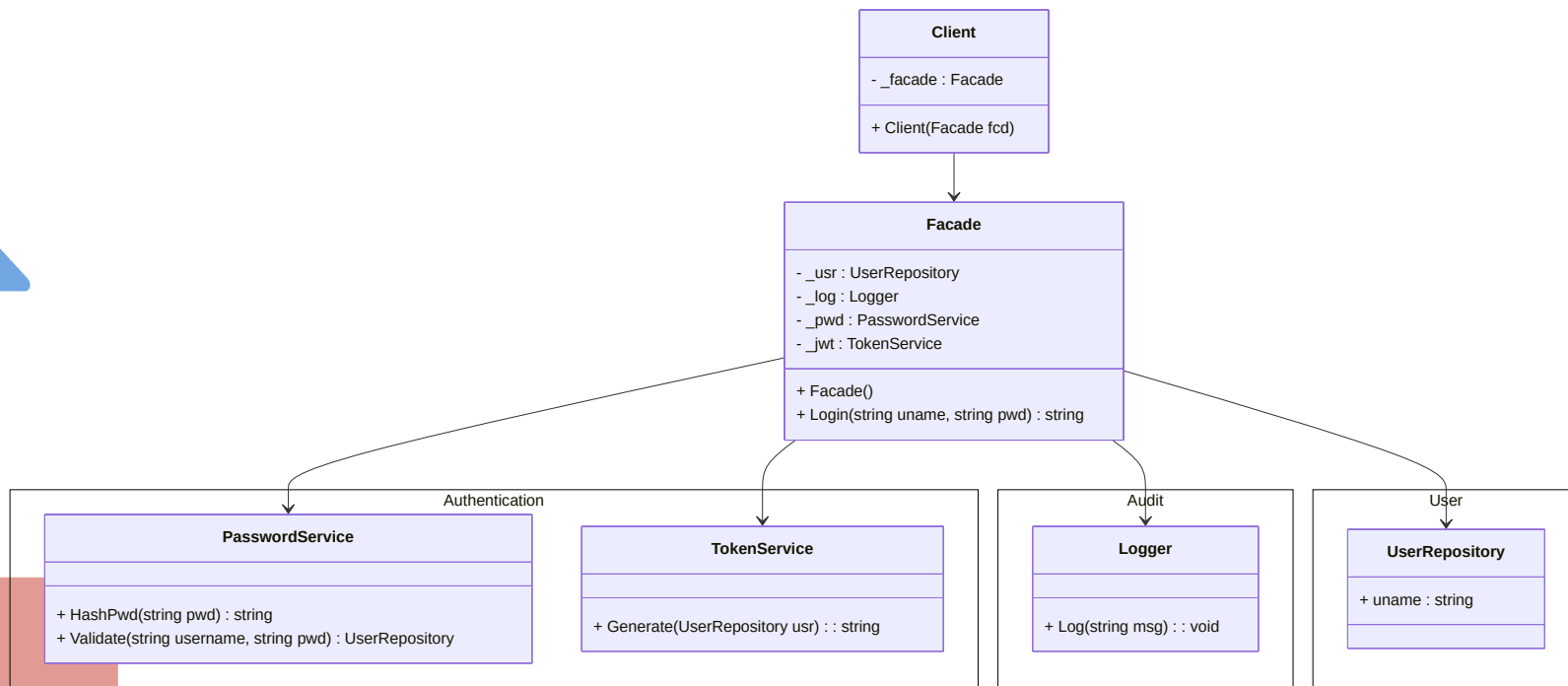
Hvis vi kan lave noget lignende:



Så skjuler vi kompleksitet, og skaber et tydeligt interface.

LØSNING

DIAGRAM



LØSNING

CLIENT + SUBSYSTEM LOGINFACADE

```
class Client
{
    static void Main()
    {
        LoginFacade facade = new LoginFacade();
        facade.Login("Stefan", "Secret#1");
    }
}
```

```
class UserRepository{
// Implementation GitHub
}
class Logger{
// Implementation GitHub
}
class PasswordService{
// Implementation GitHub
}
class TokenService{
// Implementation GitHub
}
```

```
class LoginFacade
{
    private UserRepository? _usr = null;
    private Logger _log = Logger.GetInst();
    private PasswordService _pwd = PasswordService.Empty;
    private TokenService _jwt = TokenService.Empty;
    public string Login(string uname, string pwd)
    {
        string tmp = "";
        pwd = _pwd.HashPwd(pwd);
        _usr = _pwd.Validate(uname, pwd);
        tmp = _jwt.Generate(_usr);
        if (tmp != "")
        {
            _log.Log("User: " + _usr.uname + " has succesful logged in");
        }else
        {
            _log.Log("User: " + _usr.uname + " has unsuccessful logged in");
        }
        return tmp;
    }
}
```


LØSNING

FACADE ER IKKE

- Factory (opretter ikke objekter)
- Adapter (oversætter ikke interface)
- Singleton (styrer ikke antal instanser)

Facade:

- samler kompleksitet bag ét simpelt interface

HVORNÅR BRUGER VI FACADE?

Facade bruges når:

- Et system er komplekst
- Vi ønsker et simpelt interface
- Klienten ikke skal kende subsystemer
- Vi vil reducere kobling



OPGAVE 1

TAG UD GANGSPUNKT I KODEN PÅ GITHUB

Systemet skal kunne persistere data gennem flere forskellige teknologier. Frontend eller applikationslaget må ikke kende til detaljerne i disse teknologier. Der ønskes derfor én samlet facade, som skjuler kompleksiteten. Facaden skal være uafhængig af persisteringsteknologi, læse og skrive.

- 
1. Opdatere klassediagram med den nye facade
 2. Opdatere koden med den nye facade

NÆSTE GANG

